SC3020 Database System Principles
Project 1 Report

Group 7

Name	Matriculation No.	Contributions
Muhammad Firdaus Bin Azizan	U2121149J	Task 1, Sections 2.5
Mohamed Ali Mohamed Umar	U2121706D	Task 3, Sections 3.2.2
Wong Yi Xian	U2121401C	Task 2, Sections 3.3.2
Cheong Yu Qing, Crystal	U2121124A	Task 1, Sections 1, 2.1-2.4
Choo Yi Ken	U2240710B	Task 1, Sections 2.5, 3.1-3.2.1, 3.3.1, 4-5

SC3020 Database System Principles

Project 1 Report 1
1. Introduction 3
1.1 Project Overview 3
1.2 Project Implementation 3
2. Design of the Storage Component 4
2.1 Fields 4
2.2 Record Serialisation 4
2.3 Block Storage 5
2.4 Retrieval of Records 5
2.5 Additional Design Considerations 6
2.5.1 Spanned vs Unspanned 6
2.5.2 Sequencing vs Non-sequencing 7
2.5.3 Fixed Length vs Variable Length 8
3. Design of the Indexing (B+ Tree) Component 9
3.1 Implementation 9
3.2 Main functions of B+ Tree 9
3.2.1 B+ TreeNode 9
3.2.2 B+ Tree 10
3.3 Additional Design Considerations 11
3.3.1 Parameter n of B+ Tree 11
3.3.2 Duplicate Keys 11
4. Results 12
4.1 Task 1 12
4.2 Task 2 13
4.3 Task 3 14
5. Appendix 15
5.1 Class Diagram 15

1. Introduction

1.1 Project Overview

This project focuses on developing and implementing two key components of a database management system: the storage component and the indexing component. The primary objective of the storage component is to manage the efficient storage of NBA game data on disk, ensuring that the records are organised into blocks and written to files for persistent storage. The secondary goal involves the indexing component, which will utilise a B+ Tree to enable quick retrieval of records based on a specific key attribute, namely the field goal percentage for home teams (`FG_PCT_home`). The scope of this report covers only the storage component, detailing the design and implementation of how NBA game records are structured, serialised, stored into blocks, and later retrieved.

1.2 Project Implementation

The project was implemented using Java and structured into multiple packages and classes that encapsulate specific functionalities. The key components involved are:

- `src/Main.java`: Coordinates the process of reading data, serialising records, organising them into blocks, and writing those blocks to disk.
- `src/utils/FileUtils.java`: Responsible for reading the data from a text file and converting the data into record objects.
- `src/storage/Block.java`: Represents the structure of a block and manages the addition of records, block serialisation, and block writing.
- `src/storage/Record.java`: Defines the structure of an individual NBA game record, including methods for serialising and deserializing the record data.
- `src/bplustree/BPlusTree.java`: Represents the structure of a B+ Tree and includes methods for inserting keys into nodes, splitting nodes and printing B+ Tree for debugging.
- `src/bplustree/BPlusTreeNode.java`: Defines the structure of a B+ Tree Node.

2. Design of the Storage Component

2.1 Fields

The NBA game data is parsed into individual records, each representing the statistics of a game. The fields for each record, their types, and descriptions are summarised below:

Field Name	Type	Description
`gameDate`	Date	The date when the game took place, stored as a long.
`teamIdHome`	int	The unique identifier of the home team.
`ptsHome`	int	The total points scored by the home team.
`fgPctHome`	float	The field goal percentage of the home team.
`ftPctHome`	float	The free throw percentage of the home team.
`fg3PctHome`	float	The three-point field goal percentage of the home team.
`astHome`	int	The number of assists made by the home team.
`rebHome`	int	The number of rebounds made by the home team.
`homeTeamWins`	boolean	A boolean value indicating whether the home team won.

These fields are first parsed from the input file and later serialised into a binary format for storage on disk.

2.2 Record Serialisation

The total size for each record block would be 39 bytes. The implementation of serialisation is provided below:

Field Name	Type	Size
`gameDate`	Date	10 Bytes
`teamIdHome, ptsHome, astHome, rebHome`	int	4 Bytes
`fgPctHome, ftPctHome, fg3PctHome`	float	4 Bytes
`homeTeamWins`	boolean	1 Byte

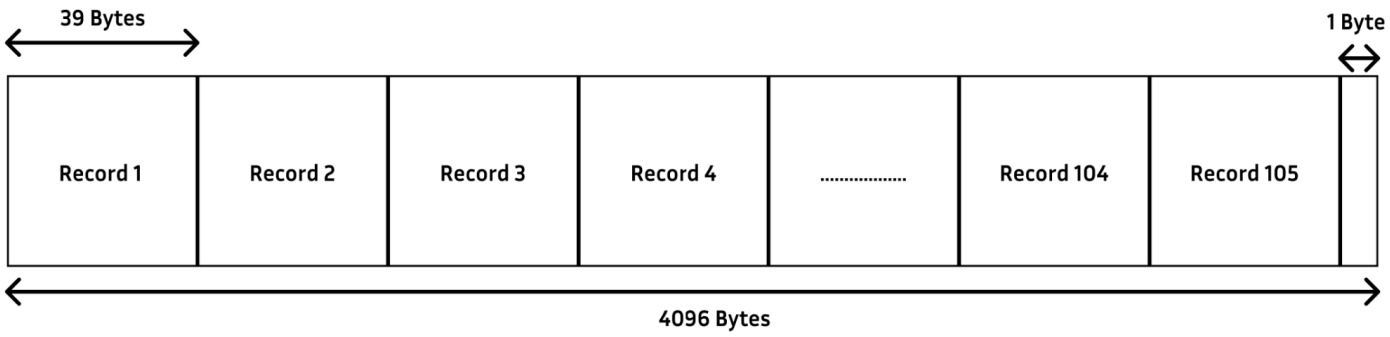
2.3 Block Storage

The project employs a **block-based storage system** where multiple records are grouped into blocks. Each block has a predefined **block size** based on the system's disk settings (determined via Java's `FileStore` API). The `Block` class manages both the creation and the storage of records into these blocks. Key elements of block storage include:

- **Block Size**: The size of the block is dynamically determined by the system's file store settings to ensure efficient I/O operations.
- **Records per Block**: Each block can hold multiple records, but the number of records is determined by the block size and the serialised size of each record (39 bytes per record). Once the block is full, it is serialised and written to a file.

Each block is written as a separate file on disk, ensuring that no block exceeds the available block size. The naming convention for these block files is `nba_game_block_<blockNumber>.dat`, and the data is written using Java's `FileOutputStream`.

For example: The block size is 4K Bytes (4096 Bytes) and record size is 39 Bytes



2.4 Retrieval of Records

To retrieve the records from disk, the system reads the serialised block files and reconstructs the original records using the `deserialize()` method of the `Record` class. This deserialization process involves:

1. **Reading Block Files**: The `Block.readBlockFromFile()` method reads the serialised data from a block file into memory. The data is stored in a `ByteBuffer`, which is then used to extract individual records.
2. **Reconstructing Records**: The byte data for each record is deserialized by converting the byte array back into the original `Record` object. For example, the `long` representing the date is converted back into a `Date` object, and the integers and floats are restored to their original values.
3. **Efficient Access**: The retrieval process is designed to allow efficient access to individual records by processing the block files one by one. This approach ensures that the system can retrieve and process large datasets with minimal memory overhead, as only one block is read into memory at a time.

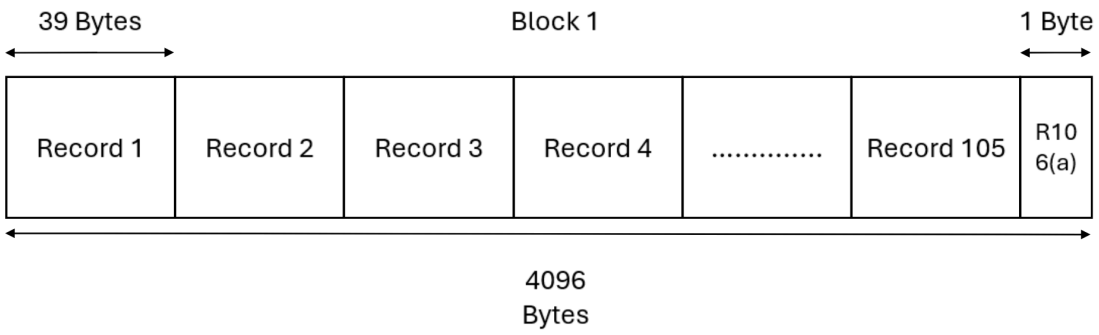
2.5 Additional Design Considerations

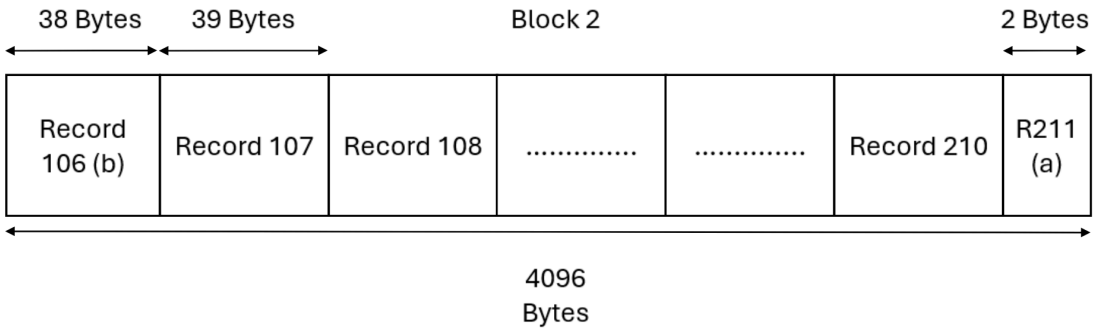
2.5.1 Spanned vs Unspanned

- If records are **spanned**, they can be split across multiple blocks when space remains. This ensures more efficient use of storage space as the block can still store part of the record when it's near full capacity.
- If records are **unspanned**, they cannot be split across blocks when there is remaining space present. Therefore, it could lead to wasted space at the end of the blocks when records do not fit perfectly. However, this reduces the complexity of the record retrieval process since records are not split into multiple chunks.

Since we fixed the block size to be 4096 Bytes and our record size is 39 Bytes, each block will contain at least 105 records with 1 byte remaining in each block. Our group chose to use the unspanned design because the wasted space in each block is only 1 byte, which is negligible, and thus reducing the complexity of the record retrieval process.

For example: If we were to use a spanned design instead, the blocks would look like:





As shown in the block diagrams above, spanned records are stored across multiple blocks, which means that retrieving a single record requires accessing more than one block. Retrieving record 106 would mean we have to access Block 1 and Block 2.

When comparing it to just using a spanned design, we are only required access to one block to retrieve the record as each record will only be in one block.

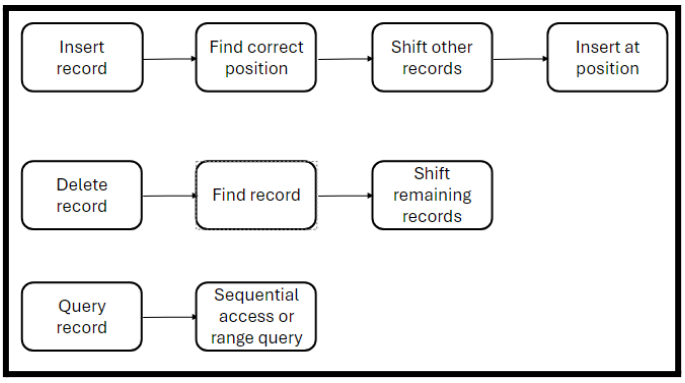
2.5.2 Sequencing vs Non-sequencing

- Records are stored in **sequencing** order of some key attribute means records are arranged in ascending or descending order with respect to the specified key attribute
- Records are stored in **non-sequencing** order means records are stored in the order they are read, without involving any other extra arrangements on order.

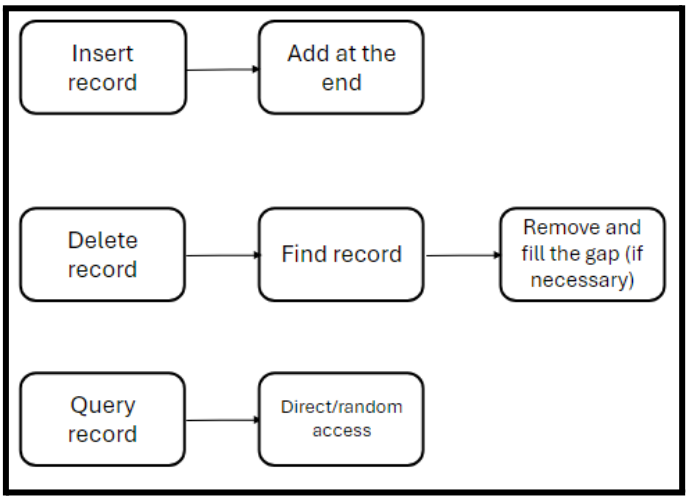
Our group implemented the project by keeping records in the blocks in a non-sequenced manner. This is to increase the speed of manipulation of records (insertion/deletion).

Operation	Sequenced Storage	Non-sequenced Storage
Insertion	Slow (requires sorting)	Fast (just appends to end)
Deletion	Slow (shifting needed)	Fast (no shifting needed)
Access Time	Faster for range queries	Faster for random access

For Sequenced Storage:



For Non-Sequenced Storage:



As shown by the visual elements above, the manipulation of the records is much faster when non-sequenced storage is used.

2.5.3 Fixed Length vs Variable Length

- For **fixed-length** storage, every record occupies the same amount of space, which requires all its attributes within the record, are of fixed size (has a defined maximum length).
- For **variable-length** storage, records may not occupy the same amount of space due to having attributes that are of variable-length data type.

Our group implemented the project using fixed-size storage because it is more consistent in size, which makes the memory allocation process more predictable and the retrieval process can be more efficient. Moreover, the dataset `games.txt` that we are using consists of records of mainly static data, which is better handled using fixed-length storage compared to variable-length storage.

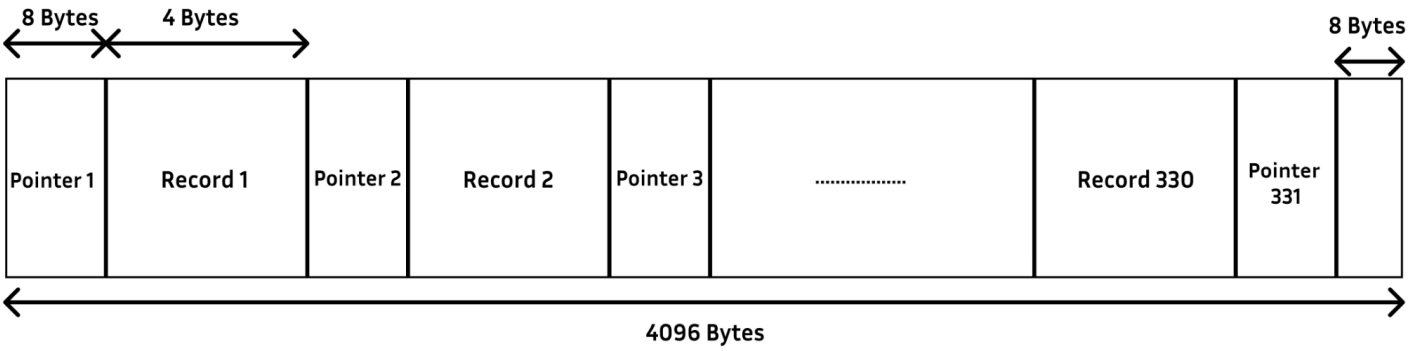
A table to justify why we selected fixed length storage over variable length storage:

Feature	Fixed-Length Storage	Variable Length
Memory Allocation	Simple and predictable. No fragmentation.	Complex, it leads to fragmentation and wasted space.
Insertion Speed	Fast (new records fit easily at the end)	Slower (when shifting or reordering is required)
Retrieval Speed	Faster (record size is predictable as it is fixed)	Slower (may require multiple reads or location adjustments)
Complexity	Low (easy to manage and allocate memory).	High (complex record handling and management).

3. Design of the Indexing (B+ Tree) Component

3.1 Implementation

To implement the B+ Tree, two classes are introduced, namely `BPlusTree` and `BPlusTreeNode`. The BPlusTree is constructed using BPlusTreeNode objects as tree nodes with a parameter `maxNumberOfKeys` to denote the n parameter of the B+ Tree Node.
The design of a B+ Tree Node is as follows:



3.2 Main functions of B+ Tree

B+ Tree is implemented for indexing purposes in our project. B+ Tree benefits from other indexing methods such as conventional indexing and hash index because it supports fast search with time complexity $O(log\ n)$ since the tree is self-balanced. In addition, B+ Tree supports range queries because lead nodes are combined as a linked list structure, making range querying easy.

3.2.1 B+ TreeNode

Attributes:

`isLeaf`	Boolean flag indicating whether the Node object is a leaf node
`keys`	The list of keys stored in the Node object
`children`	For internal nodes, it contains a list of children Nodes. For leaf nodes, it contains no information.
`records`	To store the records for each duplicate key
`next`	A pointer to the next node in the linked list at level 0

Methods:

- `getNext()`
  - To retrieve the next leaf node in the B+ Tree

3.2.2 B+ Tree

Attributes:

`root`	The root node of the B+ Tree
`maxNumberOfKeys`	The maximum number of keys per node

Methods:

- `insert()`
  - Inserts a key and the associated record into the tree.
  - This method also handles the splitting of leaf nodes if the maximum number of keys is exceeded
- `findLeaf()`
  - It is called by the `insert()` method to find the correct leaf node for insertion
  - Traverses the tree from the root to find the appropriate leaf node for the given key
- `insertIntoLeaf()`
  - It is called by the `insert()` method to insert the key and record into the correct position in the leaf node
  - Scans through the list of keys in the current leaf node to find the correct position
  - It also handles duplicate keys by adding the new record to a list of records associated with that key
- `splitLeaf()`
  - It is called by the `insert()` method when the number of keys in the leaf node exceeds the maximum number of keys
  - Splits a full leaf node into two and updates the parent nodes
- `insertIntoParent()`
  - It is called by the `splitLeaf()` method when a node splits and the resulting split key needs to be inserted into the parent node.
  - It is also called by the `splitInternal()` method for the same reason.
  - This method also checks if further splits are needed
- `splitInternal()`
  - It is called by the `insertIntoParent()` method when internal nodes need splitting.
  - This method is used when an internal node exceeds the maximum number of keys.
  - Splits the node into two internal nodes and promotes the middle key to the parent node.
  - May trigger recursive splits upwards toward the root.
- `findParent()`
  - Called by the `insertIntoParent()` method to find a parent node of a given node
  - This method recursively finds the parent of a specific child node by traversing the tree.

3.3 Additional Design Considerations

3.3.1 Parameter n of B+ Tree

The parameter `n` (max Number of Keys) is specified in the class `BPlusTree`, inside the constructor `BPlusTree(int maxNumberOfKeys)`. The parameter n is important because its magnitude will affect the B+ Tree height, as a larger n implies a shorter B+ tree. Thus, `n` is important in balancing the trade-off between node size and tree depth, affecting the performance of B+ Tree in the end.

$$n = \frac{blockSize - pointerSize}{pointerSize + keySize}$$

For our group,

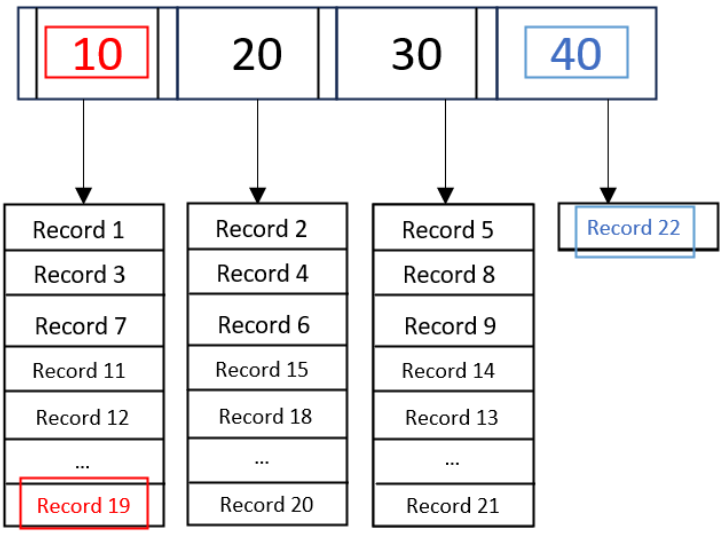
blockSize	4096 Bytes
pointerSize	8 Bytes (64 bits JVM system is assumed)
keySize (B+ Tree is indexed on `FG_PCT_home`)	4 Bytes

Thus, the value of n used in our implementation is 340.

3.3.2 Duplicate Keys

In our implementation of B+ tree, duplicate keys are handled by storing records as a list for that key value using `List<List<Record>>`. A figure example is shown below:

1) Inserting key 10, record 19
2) Inserting key 40, record 22



When inserting a duplicate key, it first checks if the key value exists:

- If the value exists, the corresponding record is appended to the list of records sharing the same key value as shown in scenario 1.
- If the value does not exist, a leaf node is created and a new list is created to store the records for that key value as shown in scenario 2.

4. Results

4.1 Task 1

The table below shows the results for the size of a record, the number of records, the number of records stored in a block, and the number of blocks for storing data. The size of a block is configured to be 4096 bytes. Note that there are a total of 99 records omitted due to incomplete information.

Size of a record	39 Bytes
Size of a block	4096 Bytes
The number of records (without NA values)	26552 records
Number of records omitted	99 records
Number of blocks for storing data	253 blocks
Number of records stored in a block (n)	First (n-1) blocks: 105
	n^{th} block: 92

The results can be verified with some basic mathematical calculations as follows:

#1

$$Let\ n\ be\ the\ number\ of\ blocks\ for\ storing\ data,\ and\ we\ have:$$
$$total\ \#\ records\ =\ (n\ -\ 1)\ \times\ max\ \#\ records\ in\ a\ block\ +\ remaining\ \#\ records$$
$$26552\ =\ (253\ -\ 1)\ \times\ 105\ +\ 92$$

#2

$$size\ of\ a\ block\ \geq\ max\ \#\ records\ in\ a\ block\ \times\ size\ of\ a\ record$$
$$4096\ \geq\ 105\ \times\ 39$$

```
-------------- Task 1 -----------------
Record Size: 39 bytes
Block Size: 4096 bytes
Number of Records Per Block: 105
Number of Records for The Last Block: 92
Total Records: 26552
Records omitted: 99
Total Blocks Written: 253
```

4.2 Task 2

The table below shows the parameter n of the B+ tree, the number of nodes in the B+ tree, the number of levels of the B+ tree, the content of the root node (only keys). The B+ tree is built on the attribute 'FG_PCT_home' as the key.

Parameter n of the B+ tree	340
Number of nodes	3
Number of levels of the B+ tree	2
Content of the root node	[0.463]

```
-------------- Task 2 -----------------
The Parameter of N in the B+ tree: 340
The Number of Nodes in the B+ tree: 3
The Number of Levels in the B+ tree: 2
The Content of root node: [0.463]
Tree has been serialized to bplustree.ser
-------------- End of Task 2 --------------
```

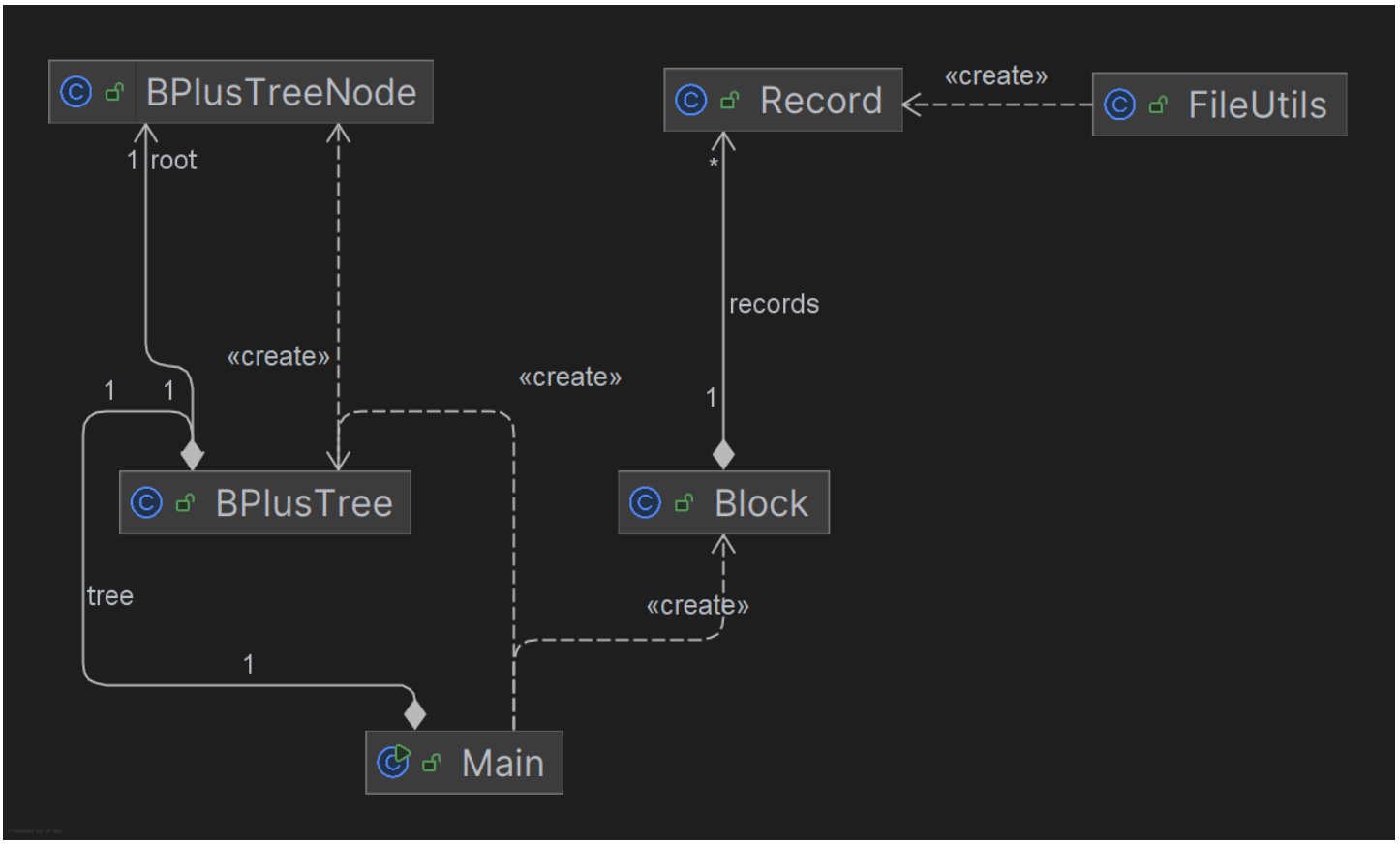
4.3 Task 3

The table below shows the average of 'FG3_PCT_home' of records returned and comparison between using B+ Tree indexing and Brute-Force Linear Scan method to access blocks of data.

Using B+ Tree	
Number of index nodes accessed	1
Number of data blocks accessed	1
Average of 'FG3_PCT_home' of records returned	0.42080122
Running time of retrieval process (nanoseconds)	123500
Using Brute Force	
Number of data blocks accessed	253
Number of records accessed	6902
Running time of retrieval process (nanoseconds)	80220200

```
-------------- Task 3 -----------------
Number of index nodes accessed: 1
Number of data blocks accessed: 1
Average FG3_PCT_home: 0.42080122
Running time of retrieval process (ns): 236600
-------------- Brute-Force Scan -----------------
Number of data blocks accessed: 253
Total records accessed: 6902
Running time of brute-force scan (ns): 81629100
```

5. Appendix

5.1 Class Diagram

Dependencies between classes defined in our project are displayed in the class diagram. Classes involved are: BPlusTreeNode, BPlusTree, Block, Record, FileUtils, Main.